Coursework 1

Computational Linear Algebra

Name: GAO YIFAN
CID: 02214998
Date: November 6, 2025

Problem 1

(a)

Below we reformulate the question into a least square problem.
Let $c = (c_0, \ldots, c_n)^T$ be the polynomial coefficients and

$$p(t_i) = c_0 + c_1 t_i + \cdots + c_n t_i^n$$

Collecting the m model values gives $p = Ac$, where the design matrix

$$A = \begin{bmatrix} 1 & t_1 & t_1^2 & \cdots & t_1^n \\ 1 & t_2 & t_2^2 & \cdots & t_2^n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & t_m & t_m^2 & \cdots & t_m^n \end{bmatrix} \in \mathbb{R}^{m\times(n+1)} \quad \forall i = 1, 2, \ldots, m$$

With positive weights $W = \operatorname{diag}(w_1, \ldots, w_m)$ where $w_i = 1/\sigma_i^2$, the weighted least squares (WLS) problem is

$$\min_{c\in\mathbb{R}^{n+1}} (Ac - y)^T W (Ac - y)$$

(Normal equations: not used in implementation though theoretically feasible) Taking the gradient gives the normal equations and because a Vandermonde Matrix is invertible, this yields:

$$A^T W A\, c = A^T W y, \qquad c = (A^T W A)^{-1} A^T W y,$$

but explicitly forming/inverting $A^T W A$ is numerically unstable (ill-conditioning), so we avoid this approach. Now we dedicated to the method using QR factorization to solve the least square problem.

(QR-based solution) Because $W > 0$, write $W^{1/2} = \operatorname{diag}(\sqrt{w_i})$ and define $B = W^{1/2}A, \quad d = W^{1/2}y$ Then WLS is the standard least–squares problem

$$\min_c \|Bc - d\|_2^2$$

Let the full QR factorization of B be $B = QR$, where $Q \in \mathbb{R}^{m\times m}$ has orthonormal columns and $R \in \mathbb{R}^{m\times(n+1)}$ is upper triangular. Then since Q is orthogonal so norm remains invariant:

$$\min_c \|Bc - d\|_2^2 = \min_c \|Q^T(Bc - d)\|_2^2 = \min_c \|Rc - Q^T d\|_2^2$$

But we proved that at the minimizer, this is equivalent to minimizing the reduced version:

$$\min_c \|\hat{R}c - \hat{Q}^T d\|_2^2$$

where $Q \in \mathbb{R}^{m\times(n+1)}$ has orthonormal columns and $R \in \mathbb{R}^{(n+1)\times(n+1)}$ is upper triangular. But this we could solve precisely and efficiently by backward substitution (since R has an inverse and is upper triangular).

(b)

(Augmented-matrix implementation) To obtain $\hat{R}$ and $\hat{Q}^T d$ simultaneously, apply an orthogonalization (e.g. modified Gram–Schmidt) to the augmented matrix so as to implicitly calculate $\hat{Q}^T d$:

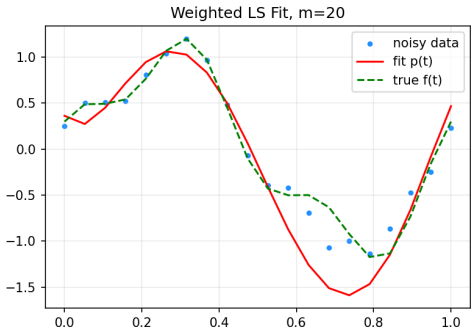
$$[\,B \mid d\,] \in \mathbb{R}^{m\times(n+2)}, \qquad [\,B \mid d\,] = Q_{\text{aug}} R_{\text{aug}} \quad \text{where } Q_{\text{aug}} = \big(q_0,\ q_1,\ \ldots,\ q_n,\ \underbrace{r_\perp/\|r_\perp\|_2}_{\text{“residual / norm”}}\ \big)$$

where the scalar $\|r_\perp\|_2$ equals the residual norm and the structure of R_{aug} is

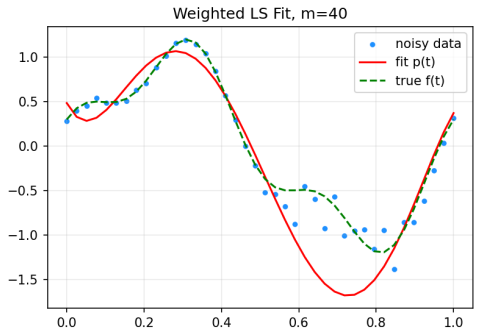
$$R_{\text{aug}} = \begin{bmatrix} \boxed{\hat{R}} & \boxed{r_d} \\ 0 & \|r_\perp\|_2 \end{bmatrix}, \quad \text{where } \hat{R} \in \mathbb{R}^{(n+1)\times(n+1)},\ r_d = \hat{Q}^T d \in \mathbb{R}^{n+1}$$

Hence the WLS solution is obtained by solving the triangular system

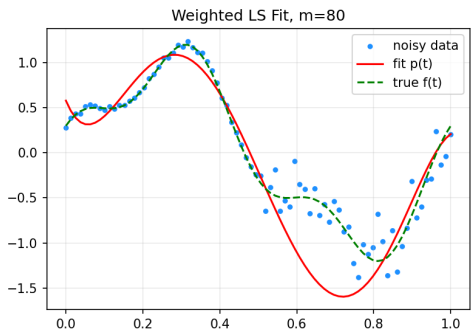
$$\hat{R}c = r_d.$$



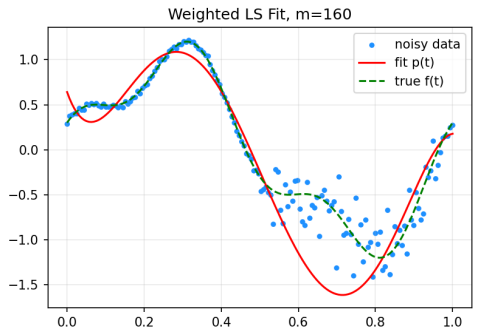
(a) WLS with m=20



(b) WLS with m=40



(c) WLS with m=80



(d) WLS with m=160

Figure 1: Weighted Least Squres fit with increasing points m=20,40,80,160 and n=5

Now to quantify the goodness of fit w.r.t changes in value of m, we test two indexes, given by the Weighted Residual Norm and rooted Mean Square Error:

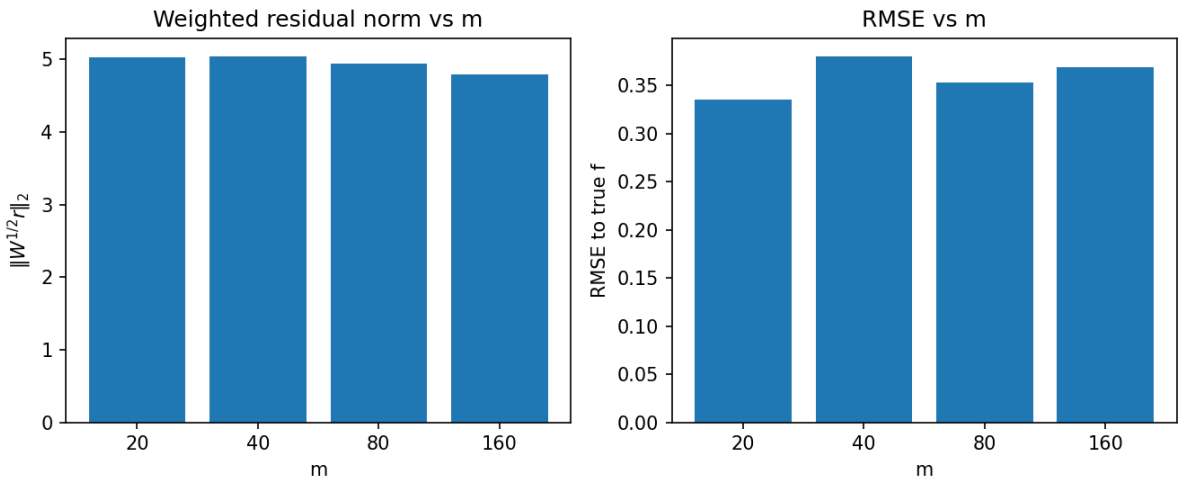


Figure 2: Test tendency in WRES and RMSE

The Weighted Residual Norm is precisely the quantity we are minimizing in (a) divided by sqrt(m), and is given by:

$$\|W^{1/2}r\|_2/\sqrt{m} = \sqrt{\frac{1}{m}\sum_{i=1}^{m}\frac{(y_i - p_i)^2}{\sigma_i^2}}$$

The Rooted Mean Squared Error is given by:

$$\text{RMSE} = \sqrt{\frac{1}{m}\sum_{i=1}^{m}(p_i - f_{\text{true}}(t_i))^2}$$

As m increases, WRES decreases slightly and stabilizes. This indicates that increasing the number of sampled points leads to a better fit in the weighted least squares model, but the improvement is not quite obvious because the polynomial degree n = 5 n=5 is fixed. In other words, the model bias (the difference between the true function and the polynomial) dominates once enough points are used to estimate the coefficients reliably.

The right panel shows the root-mean-square error (RMSE) between the fitted polynomial and the true function. The RMSE remains roughly constant, with only small fluctuations due to the randomness of the noise. This is expected because increasing m reduces the variance of the estimated coefficients, but since the polynomial model cannot perfectly represent the true function (note here we are minusing the true function, not the function with noise), the total error is dominated by the model bias. Therefore, the overall prediction error (RMSE) no longer improves significantly with larger m.

(c)

Constructing the case n = 3 without recomputing again all the coefficients.

Recall the weighted design $B = W^{1/2}A$, where $A \in \mathbb{R}^{m\times(n+1)}$ is the Vandermonde matrix and $W = \mathrm{diag}(w_i)$ with $w_i = 1/\sigma_i^2$. For $n = 5$ we have

$$B_{(5)} = \begin{bmatrix} b_0 & b_1 & b_2 & b_3 & b_4 & b_5 \end{bmatrix} = Q_{(5)}\, R_{(5)}, \qquad Q_{(5)} \in \mathbb{R}^{m\times 6},\ R_{(5)} \in \mathbb{R}^{6\times 6},$$

and from the augmented QR of $[\, B_{(5)} \mid d\,]$ we also obtain

$$r_d^{(5)} = Q_{(5)}^T d \in \mathbb{R}^6.$$

To compute the degree–3 fit, observe that $B_{(3)}$ consists of the first four columns of $B_{(5)}$:

$$B_{(3)} = \begin{bmatrix} b_0 & b_1 & b_2 & b_3 \end{bmatrix} = B_{(5)}(:, 1{:}4).$$

Since $B_{(3)}$ consists of the first 4 columns of $B_{(5)}$, the corresponding parts of the QR factors are exactly the QR factors of $B_{(3)}$:

$$Q_{(3)} \;=\; Q_{(5)}(:, 1{:}4), \qquad R_{(3)} \;=\; R_{(5)}(1{:}4,\ 1{:}4), \qquad r_d^{(3)} \;=\; r_d^{(5)}(1{:}4).$$

Hence the weighted least–squares solution for $n = 3$ is obtained without recomputing QR factorization by solving the original upper–triangular system

$$R_{(3)}\, c^{(3)} = r_d^{(3)} \quad \Longrightarrow \quad c^{(3)} = R_{(3)}^{-1} r_d^{(3)}$$

(via backward substitution)

Remark The argument holds for either modified Gram–Schmidt or Householder QR: for any QR of $B_{(5)}$, the leading 4×4 block of $R_{(5)}$ and the first four columns of $Q_{(5)}$ form the reduced QR of $B_{(3)}$, and the first four entries of $Q_{(5)}^T d$ give the right–hand side $r_d^{(3)}$.

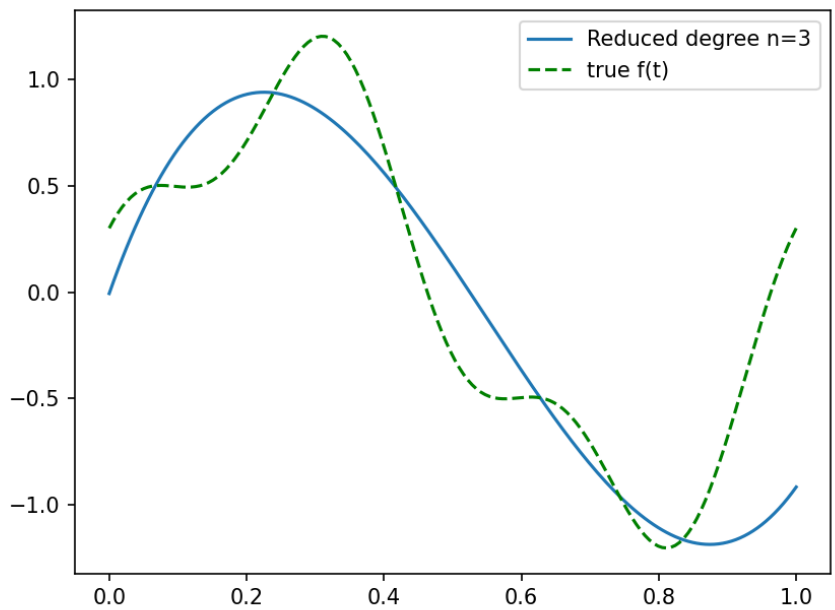


Figure 3: Reduced Order n=3 Fitted Poly to WLS

(d)

Extending from n = 5 to n = 7 without recomputing the upper triangular system: Let $B = W^{1/2}A$ and $d = W^{1/2}y$ as before. From part (b) (with $n = 5$) we already have the reduced QR of $B_{(5)} \in \mathbb{R}^{m\times 6}$:

$$B_{(5)} = Q_{(5)}R_{(5)}, \qquad Q_{(5)} \in \mathbb{R}^{m\times 6},\ \ R_{(5)} \in \mathbb{R}^{6\times 6}, \qquad r_d^{(5)} = Q_{(5)}^T d \in \mathbb{R}^6.$$

For $n = 7$ the weighted design is $B_{(7)} \in \mathbb{R}^{m\times 8}$, whose first six columns coincide with $B_{(5)}$. We construct the QR of $B_{(7)}$ by reusing the factors of $B_{(5)}$ and orthogonalizing only the two new columns that were just added:

Step 1 Fill in the Known Blocks Define

$$Q_{(7)} = \begin{bmatrix} Q_{(5)} & 0 \end{bmatrix} \in \mathbb{R}^{m\times 8}, \qquad R_{(7)} = \begin{bmatrix} R_{(5)} & 0 \\ 0 & 0 \end{bmatrix} \in \mathbb{R}^{8\times 8}.$$

Thus we copy

$$Q_{(7)}(:, 1{:}6) = Q_{(5)}, \qquad R_{(7)}(1{:}6, 1{:}6) = R_{(5)}.$$

Step 2 Extend QR for the two new columns For $k = 6, 7$ let b_k be column $k{+}1$ of $B_{(7)}$. Apply (classical or modified) Gram–Schmidt to the existing orthogonal basis:

$$r_{1:k,\,k} = Q_{(7)}(:, 1{:}k)^T b_k, \qquad v_k = b_k - Q_{(7)}(:, 1{:}k)\, r_{1:k,\,k},$$

$$r_{kk} = \|v_k\|_2, \qquad q_k = \frac{v_k}{\|v_k\|_2}.$$

Write back to the factors:

$$Q_{(7)}(:, k{+}1) = q_k, \qquad R_{(7)}(1{:}k,\ k{+}1) = r_{1:k,\,k}, \qquad R_{(7)}(k{+}1,\ k{+}1) = r_{kk}.$$

Step 3 Solve Upper Triangular System Now we successfully obtain the form again

$$r_d^{(7)} = Q_{(7)}^T d \in \mathbb{R}^8,$$

and solve the upper-triangular system by backward substitution

$$R_{(7)}\, c^{(7)} = r_d^{(7)}$$

Evaluation on a larger grid and comparison. We use a denser set of points for evaluation only, not on fitting the coefficients where we keep m = 20 fixed throughout! Typically, $n = 3$ underfits, $n = 5$ captures the main trend, while $n = 7$ may yield little improvement but avoids overfitting in the high-variance region thanks to the weighting (larger variance implies smaller weight). The weighted least squares assigns smaller weights to the high–variance region, effectively suppressing high–frequency oscillations in the fit. Consequently, the degree–7 polynomial behaves similarly to the degree–5 polynomials, providing only little improvement while avoiding oscillating too much.

In contrast, when the data are less noisy and the weights become nearly equal ($W \approx I$), the problem reduces to the ordinary least squares setting. In this context, higher–degree polynomials can better approximate the true smooth function f.

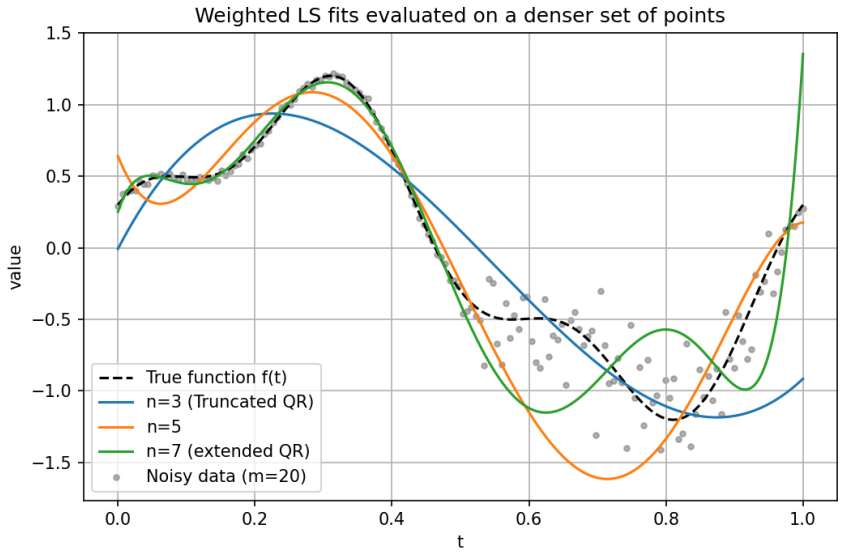


Figure 4: Contrast between WLS when n = 3, 5, 7 on denser grid

(e)

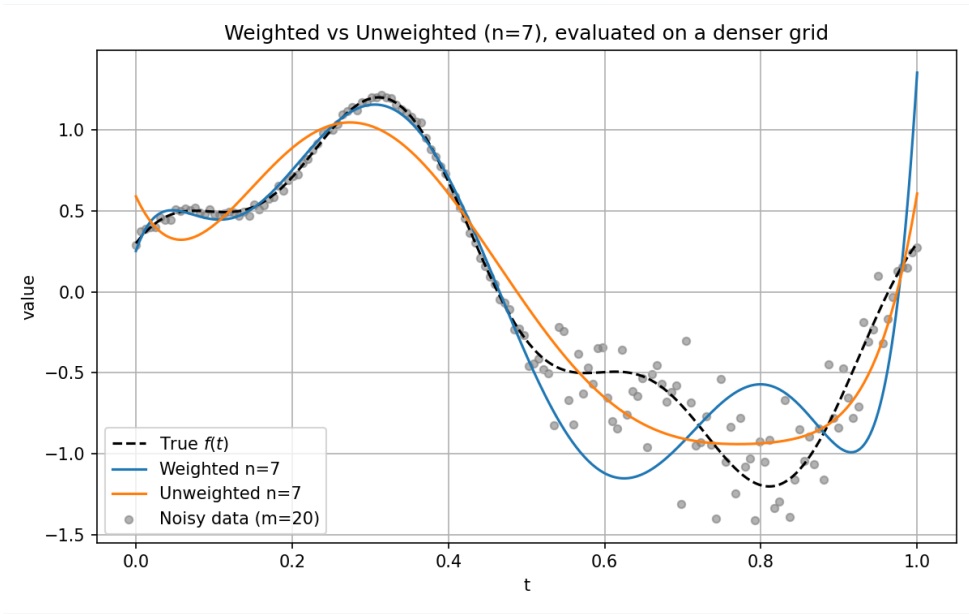


Figure 5: Comparison between WLS and OLS, taking n=7 as an example

Comparison between weighted and unweighted fits. The figure below compares the degree–7 weighted least squares (WLS) fit with the ordinary least squares (OLS) fit. In the WLS formulation, each data point is assigned a weight $w_i = 1/\sigma_i^2$, so observations with large noise variance (here for $t > 0.5$) contribute less to the minimization. Consequently, the weighted fit (blue) remains smooth and close to the true function in the high–variance region, whereas the unweighted fit (orange) overreacts to the noisy data and oscillates strongly. In the low–variance region ($t < 0.5$), both methods coincide and WLS shows a

better fit to the true function since the weights are nearly uniform. This demonstrates that the weighting scheme effectively yields a more robust fit when there are more noisy data.

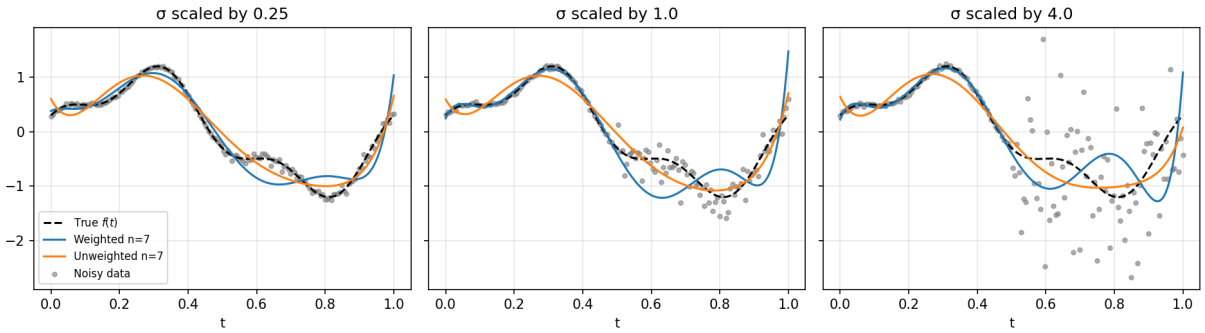


Figure 6: Comparison between WLS and OLS when increasing the noise

Effect of heterogeneous noise. In this experiment, the noise variance was increased only on the right half of the interval, otherwise all weights will be scaled equally and the optimization process in WLS remains mathematically invariant:

$$\sigma_i = \begin{cases} \sigma_0, & t_i \le 0.5, \\ s\,\sigma_0, & t_i > 0.5, \end{cases} \qquad s = 0.25, 1.0, 4.0.$$

As s increases, the Weighted Least Squares (WLS) fit assigns much smaller weights to the noisy right-hand points. Essentially, it relies on the left/middle part of the data to determine a polynomial and then extended it to $t \approx 1$. Under the Vandermonde basis, this often leads to Runge-type oscillations near the boundary. However, the unweighted Ordinary Least Squares (OLS) fit is forced to account for all points equally; the additional averaging makes the right-hand side appear smoother but with a higher bias, which may look visually smoother but is statistically suboptimal. WLS has lower bias but larger variance near the noisy region, whereas OLS shows smaller variance but higher bias. However, on the left half where the noise is small, WLS is undoubtedly more accurate since those reliable points dominate the weighted normal equations. In conclusion, WLS behaves like a cautious estimator: it sacrifices accuracy in the unreliable region to gain more accuracy in the reliable one.

Problem 2

(a)

All codes submitted to Github, with previous 3 algorithms implemented in exercises2.py and exercises3.py. I implemented Double and Triple Gram-Schmidt below.

(b)

Table 1: Orthogonality comparison of different QR factorization methods

Method	$\|Q^TQ - I\|_2$
Classical GS	73.40537695433588
Modified GS	1.3216571108941508e-10
Householder	3.3912177210886126e-15
Double GS	1.1161794843073535e-15
Triple GS	8.959572982389467e-16

(c)

Comparing the results from the five algorithms for computing QR factorizations. We see an increasing precision: Classical Gram-Schmidt » Modified Gram-Schmidt » Householder Reflections > Double Gram-Schmidt > Triple Gram-Schmidt. This is because of the following facts:

- When columns of A are nearly collinear, there are catastrophic cancellations in Classical GS, where errors will be propagated through columns, causing a gradual loss in orthogonality and a truncated error.
- Modified GS performed much better than Classical GS because every column was updated each round, and this surpressed the accumulation of projection errors. However, this algorithm still seems to perform non-ideally in case of very ill matrices.
- Double GS did another round of orthogonalization after the first round for each column. This reduced the error within machine epsilon.
- Triple GS is just a further update for each column, doing orthogonalization for another time. But the effect seemed not so obvious, because Double GS is usually enough for any matrix.
- In our experiments, DGS/TGS can occasionally achieve a slightly smaller orthogonality defect $\|Q^\top Q - I\|$ due to explicit re-orthogonalization, which polished the computed Q to nearly orthogonal matrix, but Householder remains competitive in reducing the error for $\|A - QR\|$, the optimization goals are slightly different and are reasonable.

Coursework 2

Computational Linear Algebra

Name: GAO YIFAN
CID: 02214998
Date: December 10, 2025

Problem 1

(a)

Because we are required to plot the evolution of relative error, this requires the knowledge of successive iterates of x_k. Thus I modified the original GMRES function and add an optional output of x iterations, for the purpose of this question only.
After plotting the relative error $\frac{\|e_m\|}{\|x^*\|}$ and relative residuals $\frac{\|Ax_m-b\|}{\|b\|}$, we find they differ more significantly when α grows large, but roughly remained the same trend.
Given the error vector $e_m = x_m - x^*$ and the residual $r_m = b - Ax_m$, we have

$$r_m = Ae_m.$$

Since the matrix A is SPD and orthogonally diagonalizable,

$$A = Q\Lambda Q^\top, \quad \Lambda = \mathrm{diag}(\lambda_1, \ldots, \lambda_n), \qquad 0 < \lambda_{\min} \le \lambda_i \le \lambda_{\max},$$

we obtain the following spectral bounds in the 2-norm:

$$\lambda_{\min}\|e_m\| \;\le\; \|r_m\| = \|Ae_m\| \;\le\; \lambda_{\max}\|e_m\|.$$

Equivalently,

$$\frac{1}{\lambda_{\max}}\|r_m\| \;\le\; \|e_m\| \;\le\; \frac{1}{\lambda_{\min}}\|r_m\|.$$

However, remember for the exact solution we also have $b = Ax^*$, hence using the same spectral bound in 2-norm:

$$\lambda_{\min}\|x^*\| \;\le\; \|b\| \;\le\; \lambda_{\max}\|x^*\|.$$

Therefore we obtain bounds:

- Upper bound:

$$\frac{\|e_m\|}{\|x^*\|} \le \frac{1}{\lambda_{\min}}\frac{\|r_m\|}{\|x^*\|} \le \frac{1}{\lambda_{\min}}\frac{\|r_m\|}{\|b\|}\lambda_{\max} = \kappa(A)\frac{\|r_m\|}{\|b\|}.$$

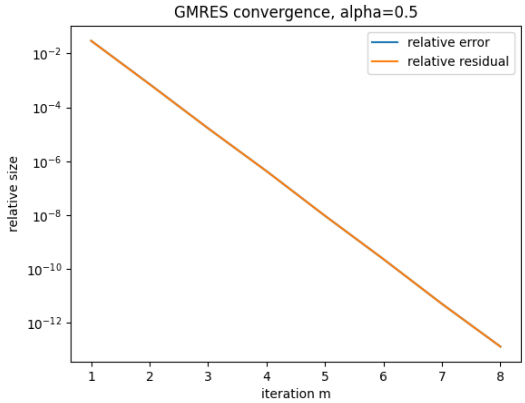
- Lower bound:

$$\frac{\|e_m\|}{\|x^*\|} \ge \frac{1}{\lambda_{\max}}\frac{\|r_m\|}{\|x^*\|} \ge \frac{1}{\lambda_{\max}}\frac{\|r_m\|}{\|b\|}\lambda_{\min} = \frac{1}{\kappa(A)}\frac{\|r_m\|}{\|b\|},$$

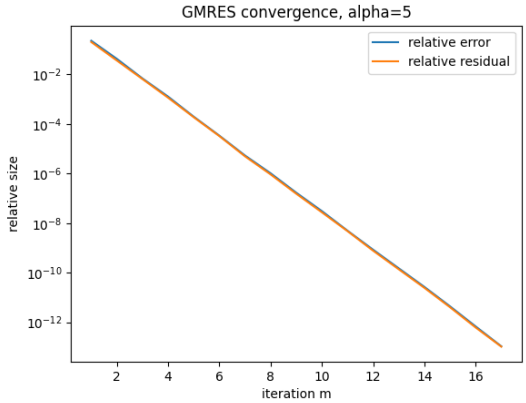
where $\kappa(A) = \frac{\lambda_{\max}}{\lambda_{\min}}$ is defined to be the 2-norm condition number of A (for an SPD matrix it equals the ratio of the extreme eigenvalues).
Thus we obtain the practically useful two-sided inequality:

$$\boxed{\frac{1}{\kappa(A)}\frac{\|r_m\|}{\|b\|} \;\le\; \frac{\|e_m\|}{\|x^*\|} \;\le\; \kappa(A)\frac{\|r_m\|}{\|b\|}}$$

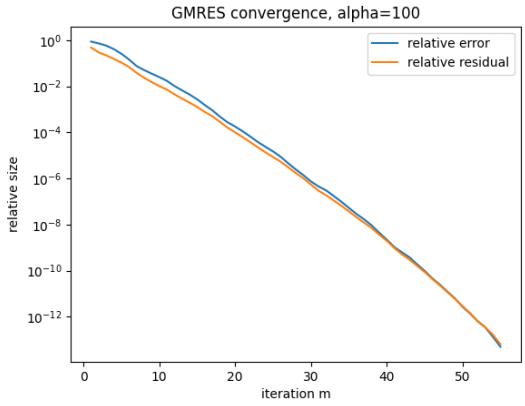
Recall how we generated the matrix A, where the eigenvalues are precisely distributed in $[5, 5+\alpha]$. So as α grows large, the conditional number grows large and the upper bound is getting looser, allowing significant difference between the convergence rate of these two quantities.



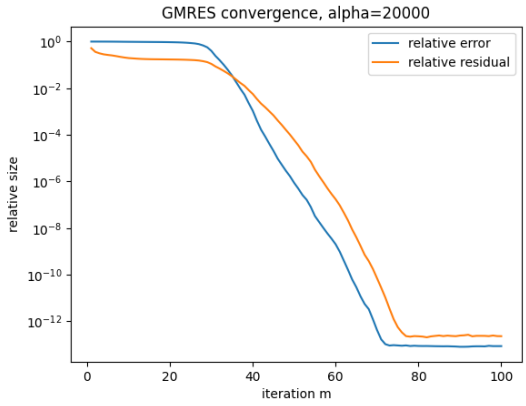
(a) Convergence when $\alpha = 0.5$



(b) Convergence when $\alpha = 5$



(c) Convergence when $\alpha = 100$

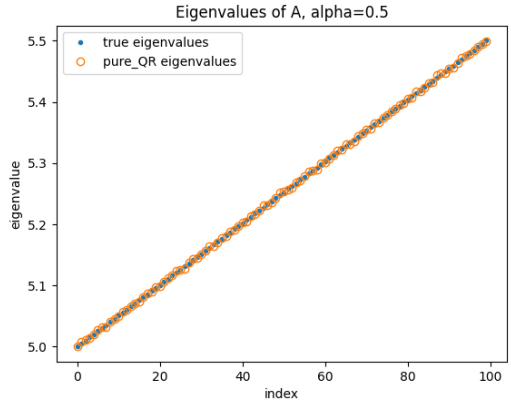


(d) Convergence when $\alpha = 20000$

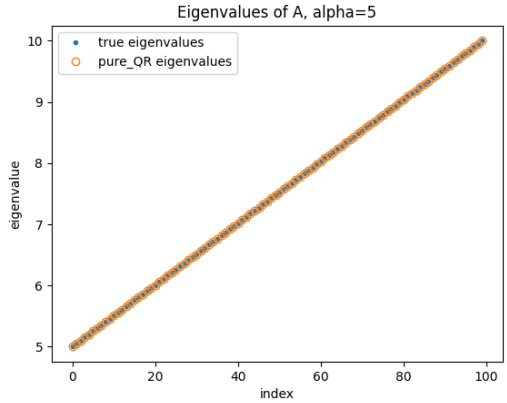
(b)

Bear in mind that Pure QR algorithm successively uses similarity transform to alter the matrix A into an upper-triangular matrix $A^{(k)} \approx T$ (Schur factorization) and T is simply diagonal in case of a Symmetric Positive Definite Matrix. The diagonal of T consists of eigenvalues of A.

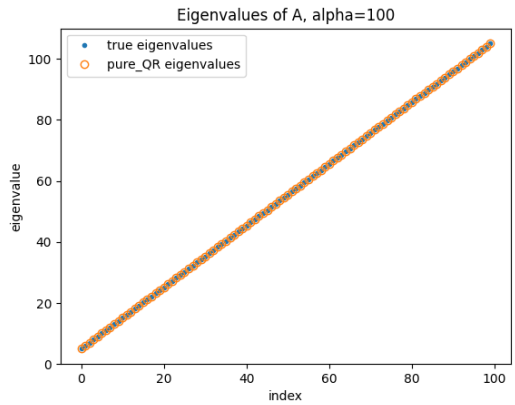
Now from the four plots above we see in each case, there is a very nice convergence of our approximated eigenvalues to the true theoretical eigenvalues, demonstrating the effectiveness of Pure QR algorithms.



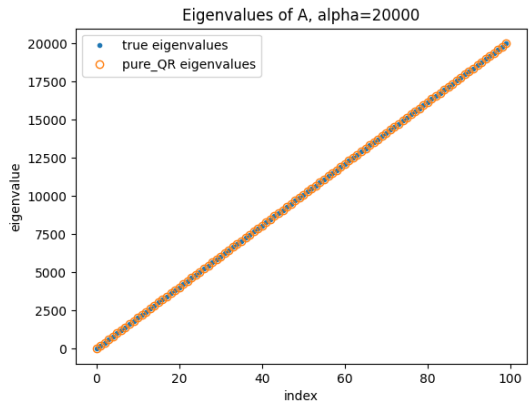
(a) Pure QR Eigenvalue Convergence when $\alpha = 0.5$



(b) Pure QR Eigenvalue Convergence when $\alpha = 5$



(c) Pure QR Eigenvalue Convergence when $\alpha = 100$



(d) Pure QR Eigenvalue Convergence when $\alpha = 20000$

(c)

Using the polynomial interpretation of GMRES, the residual after n steps can be written as

$$r_n = p_n(A)r_0 \quad \text{with} \quad p_n \in \mathcal{P}_n := \{p : \deg p \leq n,\ p(0) = 1\}$$

Using the main theorem introduced in the lecture, for a diagonalizable matrix $A = V\Lambda V^{-1}$ we have

$$\frac{\|r_n\|}{\|b\|} \leq \kappa(V) \inf_{p \in \mathcal{P}_n} \max_i |p(\lambda_i)|$$

If all eigenvalues lie in a disk $D(c, \rho)$, we can choose the specific polynomial

$$\tilde{p}(z) = \left(\frac{c - z}{c}\right)^n \in \mathcal{P}_n$$

Since $\lambda_i \in D(c, \rho)$, we have $|c - \lambda_i| \leq \rho$, and therefore

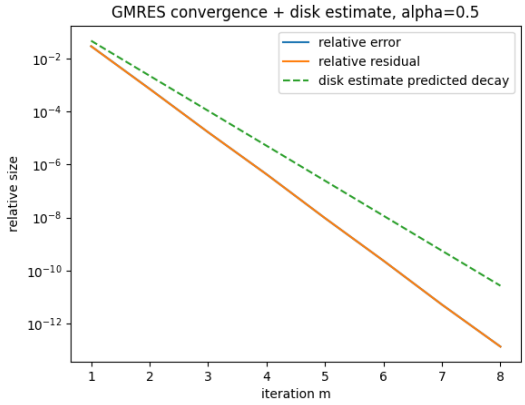
$$|\tilde{p}(\lambda_i)| = \left|\frac{c - \lambda_i}{c}\right|^n \leq \left(\frac{\rho}{|c|}\right)^n$$

Consequently, since the original inequality holds for infimum, it must hold for a specific choice of polynomial:

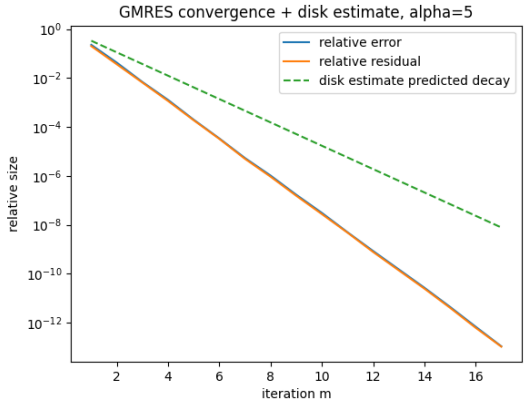
$$\frac{\|r_n\|}{\|b\|} \leq \kappa(V) \left(\frac{\rho}{|c|}\right)^n \leq \left(\frac{\rho}{|c|}\right)^n$$

since V is an orthogonal matrix. In our case it is easy to find this bound because such a disk lies on the real line, centered at $c = 5 + \alpha/2$ with radius $\rho = \alpha/2$. Calculating the ratio gives:

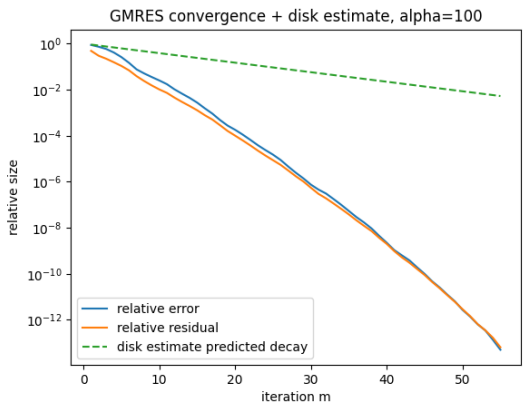
$$\frac{\rho}{|c|} = \frac{\alpha/2}{5 + \alpha/2} = \frac{\alpha}{\alpha + 10} =: q.$$



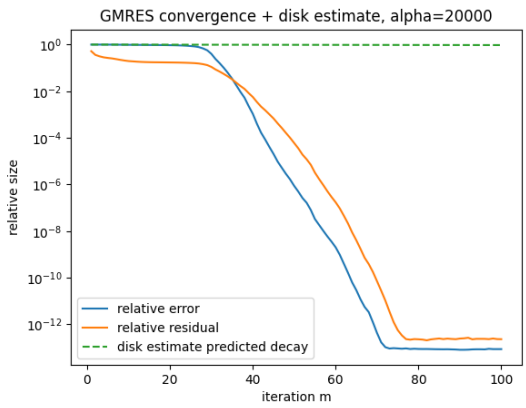
(a) Convergence with disk estimate when $\alpha = 0.5$



(b) Convergence with disk estimate when $\alpha = 5$



(c) Convergence with disk estimate when $\alpha = 100$



(d) Convergence with disk estimate when $\alpha = 20000$

We see that as $\alpha \to \infty$, the ratio tends to 1 and the convergence speed decreases, as shown by the green dashed line.

(d)

This is just performing scaling on both slides of the equation $Ax = b$, having no substantial effect in the iteration of GMRES. Moreover, we see the new eigenvalue interval becomes:

$$\left[\frac{5}{\beta}, \frac{5+\alpha}{\beta}\right]$$

Hence the new center is:

$$\tilde{c} = \frac{5/\beta + (5+\alpha)/\beta}{2} = \frac{10+\alpha}{2\beta}$$

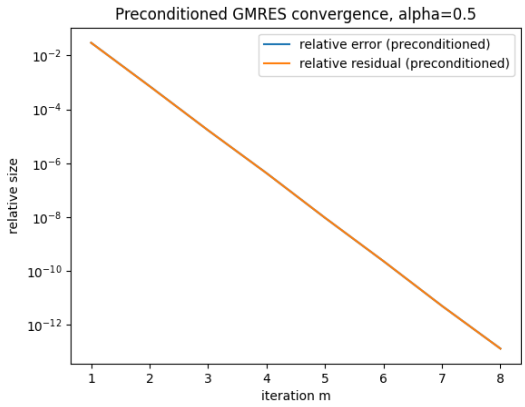
The new diameter is:

$$2\tilde{\rho} = \frac{5+\alpha}{\beta} - \frac{5}{\beta} = \frac{\alpha}{\beta}$$

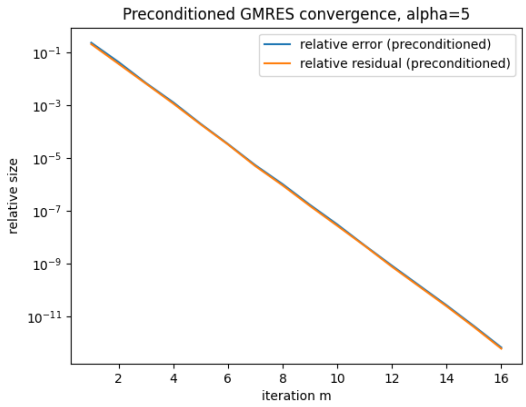
Substitution gives the ratio

$$\frac{\tilde{\rho}}{\tilde{c}} = \frac{\alpha}{10+\alpha}$$

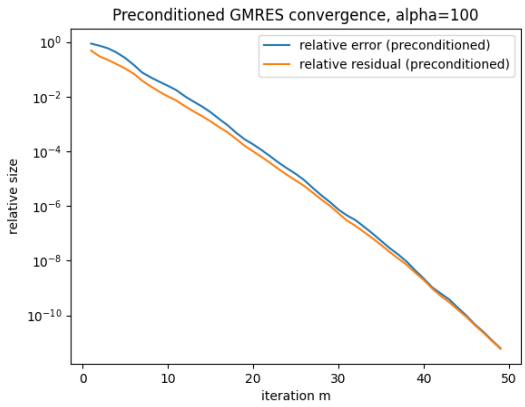
remains unchanged. So this pre-conditioner is not effective. This is somewhat counter-intuitive, one may argue the disk is shrinking, but the radius is also decaying. According to our main theorem, there is no significant improvement in the convergence.



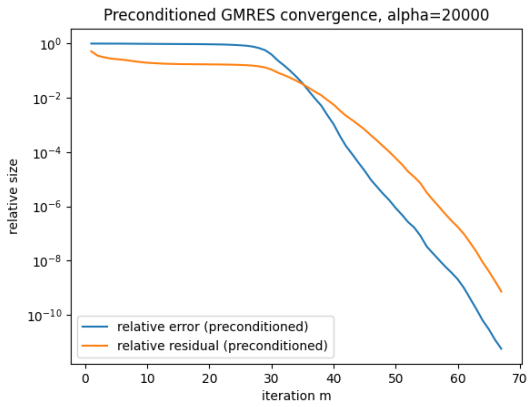
(a) Preconditioned Convergence when $\alpha = 0.5$



(b) Preconditioned Convergence when $\alpha = 5$



(c) Preconditioned Convergence when $\alpha = 100$



(d) Preconditioned Convergence $\alpha = 20000$

(e)

In this part, we use the Jacobi pre-conditioner. For the Jacobi preconditioner

$$M = \mathrm{diag}(A)$$

the left-preconditioned system is $M^{-1}Ax = M^{-1}b$. Writing $D = \mathrm{diag}(A)$, we obtain

$$A_{\text{Jacobi}} = D^{-1}A = I + D^{-1}(A - D),$$

hence all diagonal entries of A_{Jacobi} equal 1. This corresponds to a row scaling of A (dividing each row by the diagonal elements, so the new matrix has diagonal equal to 1). While the off-diagonal entries are much smaller due to the randomness of Q in the construction of A, the matrix A_{Jacobi} is significantly closer to the identity matrix than the original A. Consequently:

- its eigenvalues move toward 1,
- the spectral interval becomes much shorter,
- the eigenvalues form a much tighter cluster.

We could obtain a range for the diagonal entries d_{ii} of A. The matrix A is given by its spectral decomposition:

$$A = Q\Lambda Q^\top, \quad \Lambda = \mathrm{diag}(\lambda_1, \ldots, \lambda_n), \quad \lambda_k \in [5, 5 + \alpha].$$

The diagonal entry of A is:

$$d_{ii} = (A)_{ii} = \sum_{k=1}^{n} q_{ik}^2 \lambda_k.$$

- The sequence of coefficients $(q_{i1}^2, \ldots, q_{in}^2)$ for each row consists of non-negative coefficients that sum up to 1;
- Therefore, d_{ii} is a convex combination of the eigenvalues $\{\lambda_k\}$:

$$\lambda_{\min} \leq d_{ii} \leq \lambda_{\max}.$$

By the way we generate random matrix A, we know the non-diagonal entries will be close to 0. So for the Preconditioned Matrix A_{Jacobi}, the non-diagonal entries will be shrinked even closer to 0 while the diagonal entries remain 1. The pre-conditioner is considered good when the pre-conditioned A is close to idendity, because if close to I, then all eigenvalues λ_i are close to 1:

$$\lambda_i \approx 1.$$

Let λ_i all fall within the interval:

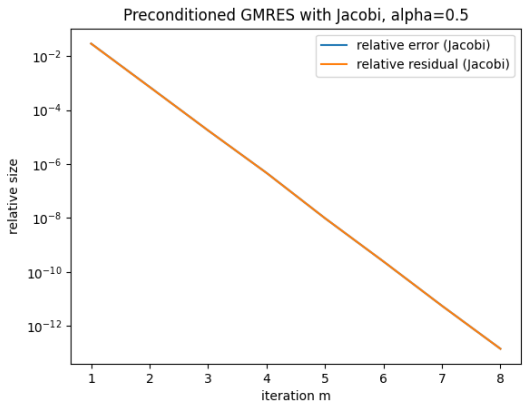
$$\Lambda(A) \subset [1 - \varepsilon, 1 + \varepsilon], \quad \varepsilon \ll 1.$$

Then we can construct a polynomial:

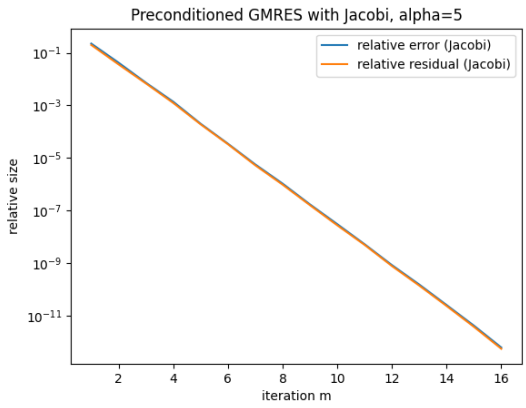
$$p_n(z) = (1 - z)^n.$$

Check its value on λ_i gives rapid convergence using our main theorem:

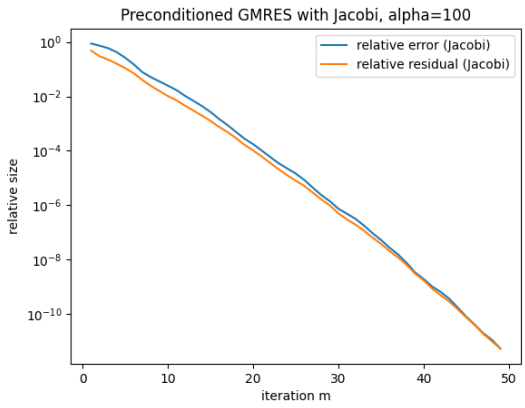
$$|p_n(\lambda_i)| = |1 - \lambda_i|^n \leq \varepsilon^n.$$



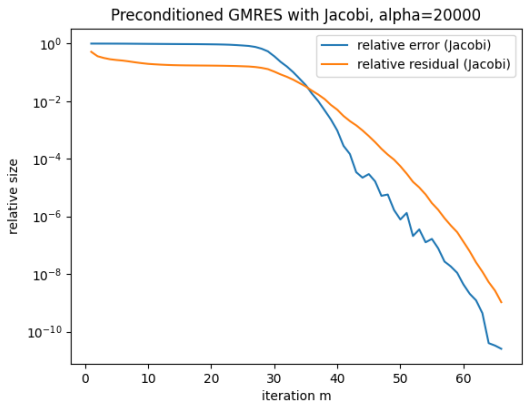
(a) Preconditioned Convergence when $\alpha = 0.5$



(b) Preconditioned Convergence when $\alpha = 5$



(c) Preconditioned Convergence when $\alpha = 100$



(d) Preconditioned Convergence $\alpha = 20000$

However, we do see that our left Jacobi pre-conditioner has limited improvement to the original system. Alternatively, we could proceed by using the Symmetric Gauss-Seidel (SGS) preconditioner introduced in the Stationary Iteration Method. The formula is given by:

$$M = (L + D)D^{-1}(D + L)^T$$

with its inverse:

$$M^{-1} = (L + D)^{-T}D(L + D)^{-1}$$

Recall why we are able to use Stationary Iteration to construct our pre-conditioner for the GMRES algorithm (We only demonstrate simple cases, such as one-step iteration). If M^{-1} approximates A^{-1} well then:

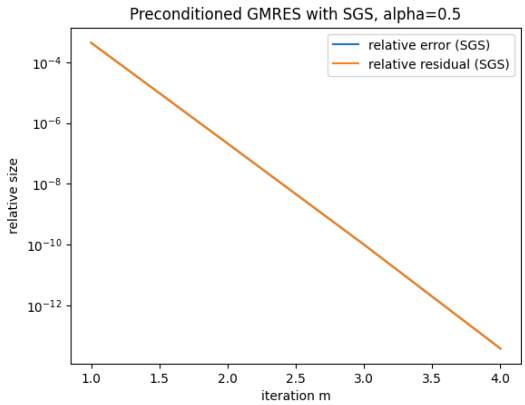
$$M^{-1}A \approx I \quad \Rightarrow \quad I - M^{-1}A \approx 0$$

And our single step iteration error $e^{k+1} = (I - M^{-1}A)e^k$ quickly drops to zero. So stationary Iteration finds a cheap approximated inverse of A such that the error converges extremely fast.

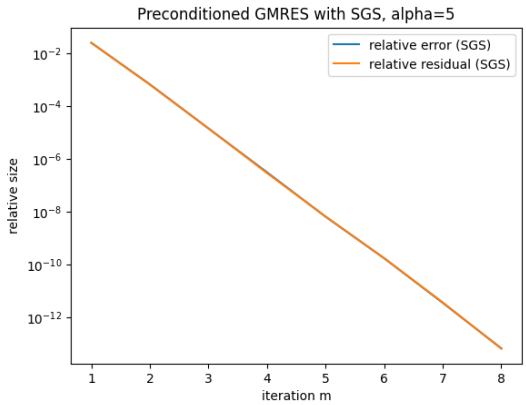
Now look at GMRES in solving the system $Ax = b$. The pre-conditioned system is:

$$M^{-1}Ax = M^{-1}b$$

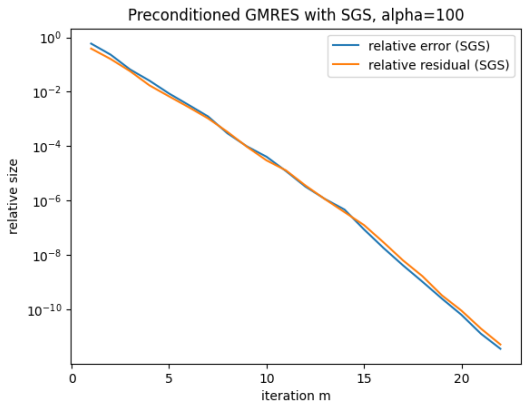
So if $M^{-1}A \approx I$ then all spectrum will be highly clustered and close to 1, then the relative residual will converege extremely fast by using our main theorem. Hence pre-conditioning of GMRES and Stationary Iteration share the common goal. Below is the convergence of the relative error and relative residual for using SGS, lesser amount of iteration is required.



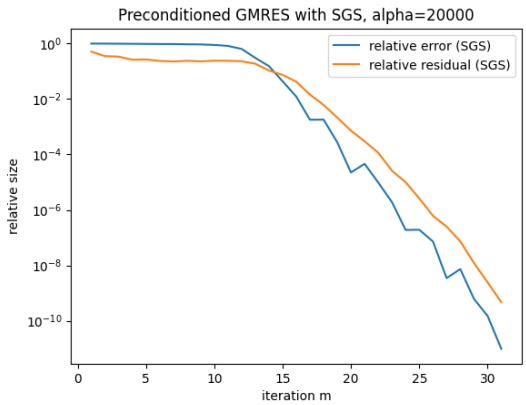
(a) Preconditioned Convergence when $\alpha = 0.5$



(b) Preconditioned Convergence when $\alpha = 5$



(c) Preconditioned Convergence when $\alpha = 100$



(d) Preconditioned Convergence $\alpha = 20000$

Problem 2

(a)

Consider the residual vector $r(\mu) = Ax - \mu x$, we could view this as a minimization problem by projecting Ax onto $\text{span}\{x\}$. To minimize $\|r(\mu)\|_2$ over all scalars μ, we want $r(\mu^*)$ to be orthogonal to the one-dimensional subspace $\text{span}\{x\}$, as the definition of orthogonal projection. i.e.

$$r(\mu^*) \perp x \quad \Longleftrightarrow \quad x^\top r(\mu^*) = 0.$$

Substituting $r(\mu^*)$ gives

$$x^\top (Ax - \mu^* x) = 0 \quad \Longrightarrow \quad x^\top Ax - \mu^* x^\top x = 0.$$

Solving for μ^* yields

$$\mu^* = \frac{x^\top Ax}{x^\top x},$$

which is precisely the Rayleigh quotient of A with respect to x.
Alternatively, we could proceed by variational calculus, simply calculating stationary points and verify that the second derivative is greater or equal to 0 (indication of maximum)

$$\phi(\mu) = (Ax - \mu x)^\top (Ax - \mu x).$$

Expanding term-wise gives:

$$\phi(\mu) = (Ax)^\top (Ax) - \mu (Ax)^\top x - \mu x^\top (Ax) + \mu^2 x^\top x.$$

Calculating derivatives with respect to μ and set it to zero recovers Rayleigh Quotient:

$$\phi'(\mu^*) = 0 \implies -2x^\top Ax + 2\mu^* x^\top x = 0 \implies \mu^* = \frac{x^\top Ax}{x^\top x}.$$

(b)

In the algorithm, I examined the convergence rate through the convergence of the residual norm, defined by $\|r_k\| := \|Ax^{(k)} - \lambda_k x^{(k)}\|$. I estimate the convergence factor by computing the successive ratio of $\|r_k\|$ and taking the average over the last 10 ratios. We know that residual norm has the same magnitude as the norm of eigenvector errors. What is the rate of convergence for eigenvector errors? According to Theorem 99 in the lecture notes, we quote it here:

Theorem 99: *If $|\lambda_1| > |\lambda_2|$ and $\|q_1^\top v_0\| > 0$, then after k iterations of power iteration, we have*

$$\|v^{(k)} - \pm q_1\| = \mathcal{O}\left(\left|\frac{\lambda_2}{\lambda_1}\right|^k\right), \quad |\lambda^{(k)} - \lambda_1| = \mathcal{O}\left(\left|\frac{\lambda_2}{\lambda_1}\right|^{2k}\right),$$

We know the ratio ρ is determined by the ratio of second largest eigenvalue over the largest eigenvalue. But calculating true eigenvalues requires preliminary insights of problem (e). We thus make the following notations:

$$\theta_j = \frac{j\pi}{2(n+1)}, \quad \lambda_j = \frac{4}{h^2}\sin^2\theta_j, \quad h = \frac{1}{n+1}.$$

Notice here the largest eigenvalue is λ_n, with λ_{n-1} the second largest one. Now we try to estimate the magnitude by using trigonometric identities and small angle approximations:

$$\theta_n = \frac{n\pi}{2(n+1)} = \frac{\pi}{2} - \frac{\pi}{2(n+1)} = \frac{\pi}{2} - \delta, \quad \delta := \frac{\pi}{2(n+1)}.$$

$$\sin\theta_n = \sin\left(\frac{\pi}{2} - \delta\right) = \cos\delta.$$

$$\sin^2\theta_n = \cos^2\delta \approx \left(1 - \frac{\delta^2}{2}\right)^2 \approx 1 - \delta^2$$

Hence, concluding the asymptotic estimates for λ_n and thus similarly for λ_{n-1}:

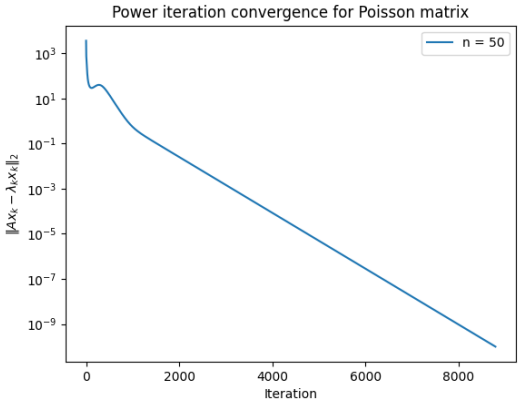
$$\lambda_n = 4h^{-2}\sin^2\theta_n \approx 4(n+1)^2\left(1 - \frac{\pi^2}{4(n+1)^2}\right) = 4(n+1)^2 - \pi^2.$$

$$\lambda_{n-1} \approx 4(n+1)^2 - 9\pi^2.$$

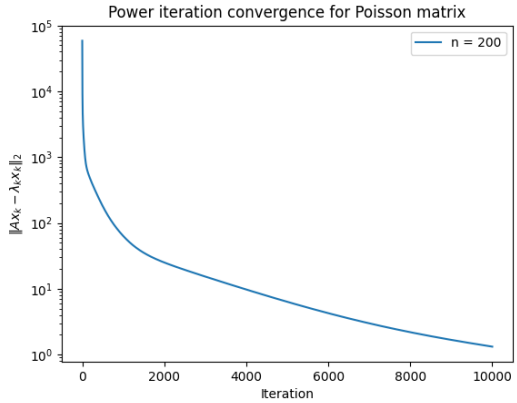
Therefore the spectral ratio is approximately 1, aligned with our computational results:

$$\rho = \frac{\lambda_{n-1}}{\lambda_n} \approx \frac{4(n+1)^2 - 9\pi^2}{4(n+1)^2 - \pi^2} \to 1$$

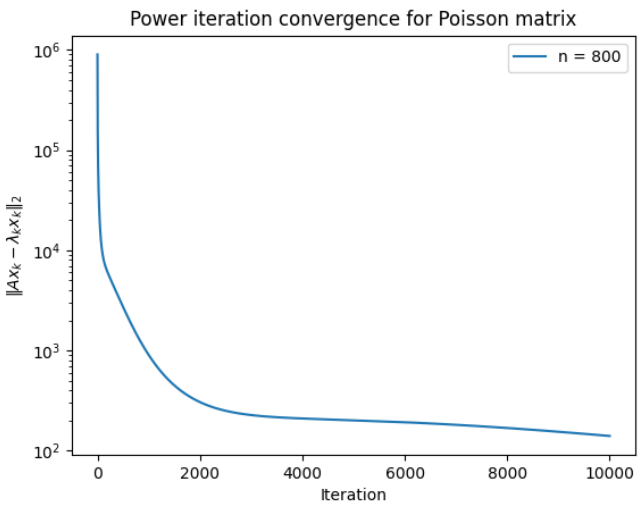
As n increases, we expect ρ to be closer and closer to 1, resulting the convergence rate to decrease.



(a) Power Iteration Convergence n=50



(b) Power Iteration Convergence n=200



(c) Power Iteration Convergence n=800

(c)

After running the shifted inverse algorithm, we found surprisingly that convergence factors for different n are roughly the same, requiring similar number of steps to converge in each case. Similar to the previous question, we would like to explore the convergence factor ρ. The eigenvalues of A are λ_j, so the eigenvalues of A^{-1} are $1/\lambda_j$. Consequently, the largest eigenvalue of A^{-1} becomes $1/\lambda_1$ (since λ_1 is the smallest eigenvalue of A). We also know that perform inverse iteration with shift $\mu = 0$ is equivalent to performing power iteration with matrix A.
From Theorem 99 we know that:

$$\rho_{\text{inv}} = \left| \frac{\text{second largest eigenvalue}}{\text{largest eigenvalue}} \right| = \frac{1/\lambda_2}{1/\lambda_1} = \frac{\lambda_1}{\lambda_2}.$$

For small j, the angle

$$\theta_j = \frac{j\pi}{2(n+1)}$$

is small, allowing the use of the small angle approximation. Thus we have,

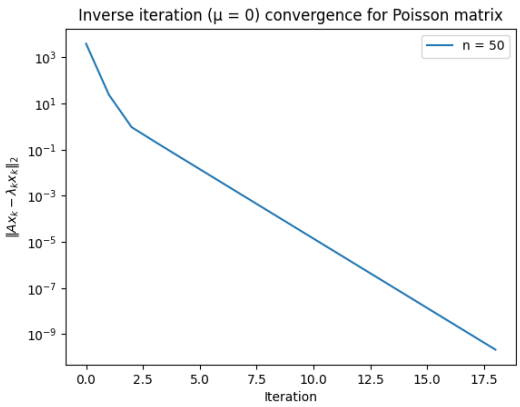
$$\lambda_j = \frac{4}{h^2}\sin^2\theta_j \approx \frac{4}{h^2}\theta_j^2 = 4(n+1)^2\left(\frac{j\pi}{2(n+1)}\right)^2 = j^2\pi^2.$$

$$\lambda_1 \approx \pi^2, \quad \lambda_2 \approx 4\pi^2.$$

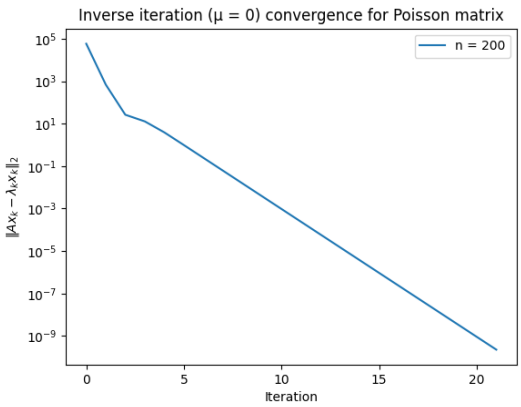
We do see a constant ratio of $\frac{1}{4}$, hence convergence rate for all n is similar. The eigenvalues satisfy $\lambda_j \approx j^2\pi^2$ for small j, showing they are strongly clustered near 0 and grow quadratically. This quadratic growth thus implies that the spacing

$$\lambda_{j+1} - \lambda_j \approx (2j+1)\pi^2$$

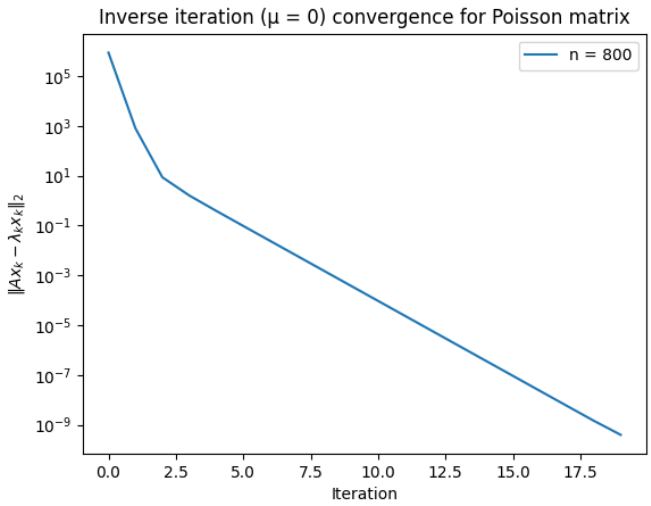
is large and, for small j completely independent of n (we require j small to satisfy condition for small angle approximation). Therefore, the ratio λ_1/λ_2 stays $\approx 1/4$ for all grid sizes.



(a) Inverse Iteration Convergence n=50



(b) Inverse Iteration Convergence n=200



(c) Inverse Iteration Convergence n=800

(d)

We could achieve the third largest eigenvalue by either using rank-1 deflation of matrices, or by using the simultaneous iteration method (initializing 3 random column vectors, left

multiply by A several times and then perform Modified Gram-Schmidt Algorithm to get the third largest eigenvector). Here we demonstrate the theory behind rank-1 deflation of SPD matrices in detail:
Let $A \in \mathbb{R}^{n\times n}$ be symmetric positive definite, with eigenvalues labeled in descending order:

$$\lambda_1 > \lambda_2 > \cdots > \lambda_n > 0,$$

and corresponding orthonormal eigenvectors $q_1, \ldots, q_n$, forming an orthonormal basis. Denote $Q = [q_1 \ \ldots \ q_n]$ and $\Lambda = \mathrm{diag}(\lambda_1, \ldots, \lambda_n)$. As a direct result from the spectral theorem for symmetric matrices:

$$A = Q\Lambda Q^\top = \sum_{j=1}^{n} \lambda_j q_j q_j^\top.$$

Define the rank-1 deflated matrix $A^{(1)} := A - \lambda_1 q_1 q_1^\top$. Now apply it to an arbitrary eigenvector q_j:

For $j = 1$, q_1 remains an eigenvector of $A^{(1)}$ but now with eigenvalue 0:

$$A^{(1)}q_1 = (A - \lambda_1 q_1 q_1^\top)q_1 = Aq_1 - \lambda_1 q_1(q_1^\top q_1) = \lambda_1 q_1 - \lambda_1 q_1 = 0.$$

For $j \geq 2$: Since $q_j \perp q_1$, i.e. $q_1^\top q_j = 0$, we have q_j is still an eigenvector of $A^{(1)}$ with the same eigenvalue λ_j:

$$A^{(1)}q_j = (A - \lambda_1 q_1 q_1^\top)q_j = Aq_j - \lambda_1 q_1(q_1^\top q_j) = \lambda_j q_j - \lambda_1 q_1 \cdot 0 = \lambda_j q_j.$$

In other words, the contribution of the largest eigenvector has been erased and we could repeatedly apply rank-1 deflation to our SPD matrix to extract the third largest eigenvalue, which will be the leading eigenvalue of $A^{(2)}$.

(e)

For shifted inverse iteration we work with the matrix $B = (A - \mu I)^{-1}$. Assume μ is chosen sufficiently close to a target eigenvalue λ_j. Then the eigenvalue of B with the largest modulus is:

$$\theta_j = \frac{1}{|\lambda_j - \mu|}.$$

The eigenvalue of B with the second largest magnitude is:

$$\max\left\{\frac{1}{|\lambda_{j-1} - \mu|}, \frac{1}{|\lambda_{j+1} - \mu|}\right\}$$

Consequently, the convergence factor (ratio of the second largest to the largest eigenvalue) of B becomes:

$$\rho_j = \frac{\max\left\{\frac{1}{|\lambda_{j-1} - \mu|}, \frac{1}{|\lambda_{j+1} - \mu|}\right\}}{\frac{1}{|\lambda_j - \mu|}} = \frac{|\lambda_j - \mu|}{\min\big(|\lambda_{j-1} - \mu|,\ |\lambda_{j+1} - \mu|\big)}$$

However, if one sets the shift equal the true eigenvalue, $\mu = \lambda_j$, then $\rho_j = 0$. Theoretically, convergence occurs in a single iteration (provided the initial vector has a nonzero

component along q_j). Numerically, due to round-off errors in floating numbers, it is usual to observe a convergence to machine epsilon within 2–3 steps.
Why does the question chose the analysis eigenvalues in the middle (i.e. $j \approx \frac{n}{2}$). We are given

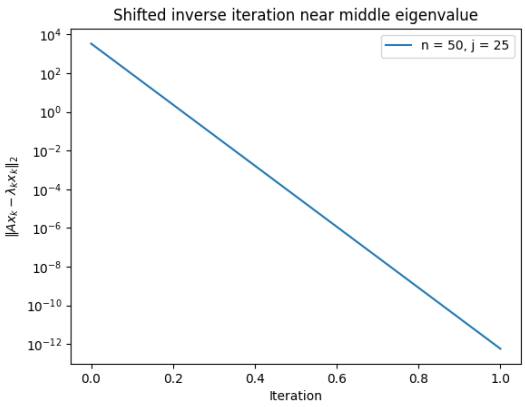
$$\lambda_j = \frac{4}{h^2}\sin^2\left(\frac{j\pi}{2(n+1)}\right), \quad h = \frac{1}{n+1},$$

which is the standard spectrum of the discrete Poisson operator on $(0, 1)$.

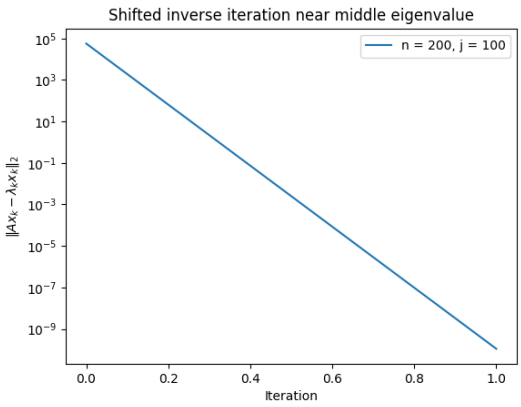
The eigenvalues increase strictly with j, but the spacing between consecutive eigenvalues $\lambda_{j+1} - \lambda_j$ is not uniform, and is in fact clustered at both ends and sparse in the middle. Let us compute this spacing:

$$\lambda_{j+1} - \lambda_j = \frac{4}{h^2}\left[\sin^2\left(\frac{(j+1)\pi}{2(n+1)}\right) - \sin^2\left(\frac{j\pi}{2(n+1)}\right)\right].$$

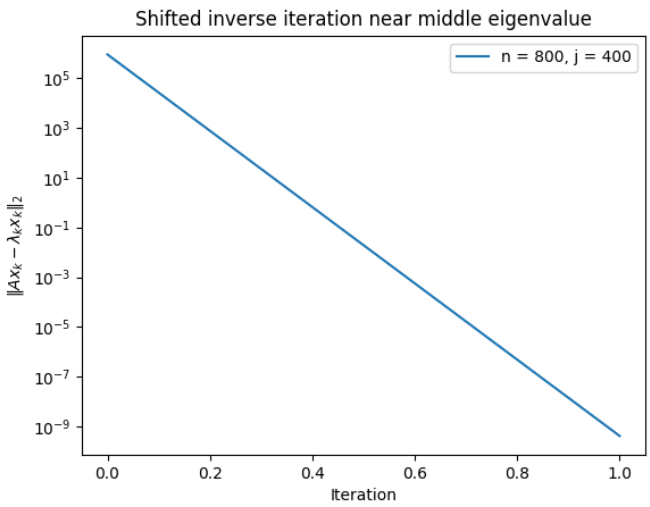
After lengthy calculations, we found the largest spacing is in the middle of all the eigenvalues. By looking at the convergence factor for this shifted inverse iteration, we see this is the most stable case.



(a) Inverse Iteration Convergence n=50



(b) Inverse Iteration Convergence n=200



(c) Inverse Iteration Convergence n=800